

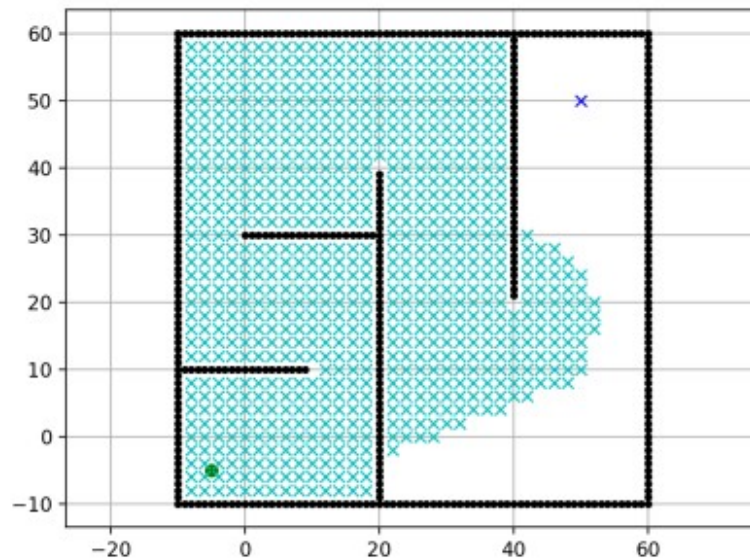
Assignment : RE605 - PATH PLANNING

Name: [4222301042] Aldon Zufar Putra Twyn

Class : Robotika – B Pagi [Semester 6]

Topic : Mempelajari Algoritma PRM dan RRT [Practice 4 & 5]

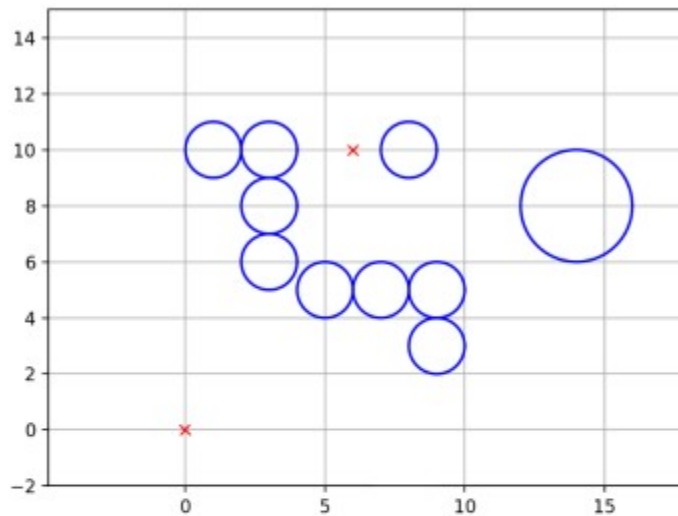
QUESTIONS PRACTICE 4 & 5



Gambar 6-4 Peta Obstacle

2. Jelaskan perbedaan antara algoritma PRM, Dijkstra's, A* dan D*?
3. Apa saja langkah-langkah utama dalam algoritma PRM dan bagaimana algoritma ini membangun graf representasi dari lingkungan robot?
4. Mengapa validasi koneksi antara titik-titik sampel penting dalam PRM, dan bagaimana algoritma memastikan koneksi yang dihasilkan adalah lintasan yang aman untuk dilalui oleh robot?
5. Bagaimana hasil simulasi program dapat membantu kita memahami kinerja algoritma PRM dalam situasi praktis, dan apakah ada cara untuk meningkatkan efisiensi atau kualitas lintasan yang dihasilkan?

Soal pertanyaan untuk Practice #4 [PRM]



Gambar 7-3 Obstacle Map RRT

3. Apa yang dimaksud dengan "ekstraksi jalur" dalam algoritma RRT, dan bagaimana prosesnya memastikan bahwa jalur yang diekstraksi adalah jalur yang memadai dari titik awal ke titik tujuan?
4. Apa perbedaan antara algoritma RRT dan PRM?

Soal pertanyaan untuk Practice #5 [RRT]

ANSWERS

Di pertemuan minggu ini kami ditugaskan untuk mengerjakan 2 tipe algoritma baru di 2 tugas practice berbeda yaitu **Practice#4** dan **Practice#5** yang telah disediakan di website learning, algoritma ini adalah algoritma **PRM** dan **RRT** atau **Probabilistic Road Map** dan **Rapidly Exploring Random Trees** dan berikut adalah hasil pengerjaan tugas saya dibawah ini.

PRACTICE - #4 [PRM]

Berbeda dengan algoritma yang sudah dipelajari sebelumnya yaitu A*, Dijkstra dan D*. PRM atau Probabilistic Road Mapping memiliki 2 fase ketika dijalankan. Sedangkan Dijkstra, A* dan D* hanya merupakan algoritma 1 fase yang bekerja dengan sistem Fixed Grid.

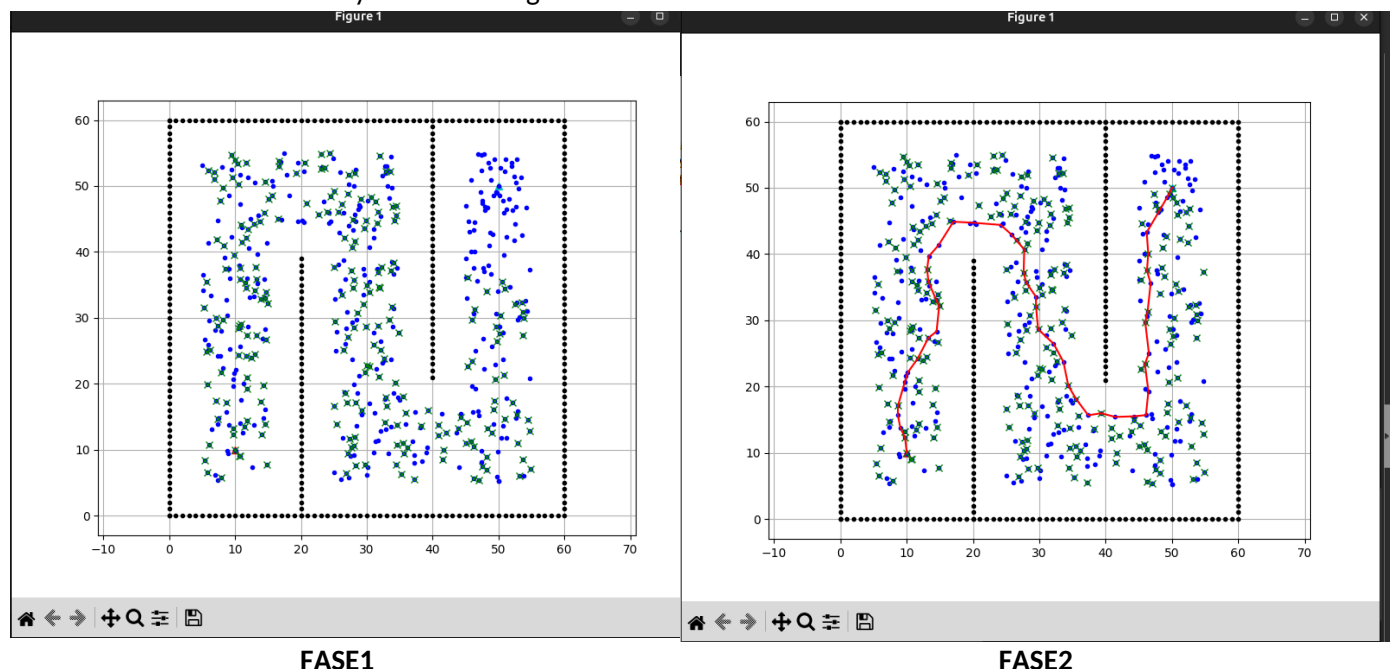
Fase 1

Di fase ini algoritma akan mengeluarkan titik acuan atau *scatter point* secara acak untuk menghindari obstacle lalu saling menghubungkan antara titik acuan terdekat yang salih terhubung tanpa halangan diantaranya yang dimana ini akan membuat semacam roadmap ketika setiap titik ke titik dihubungkan.

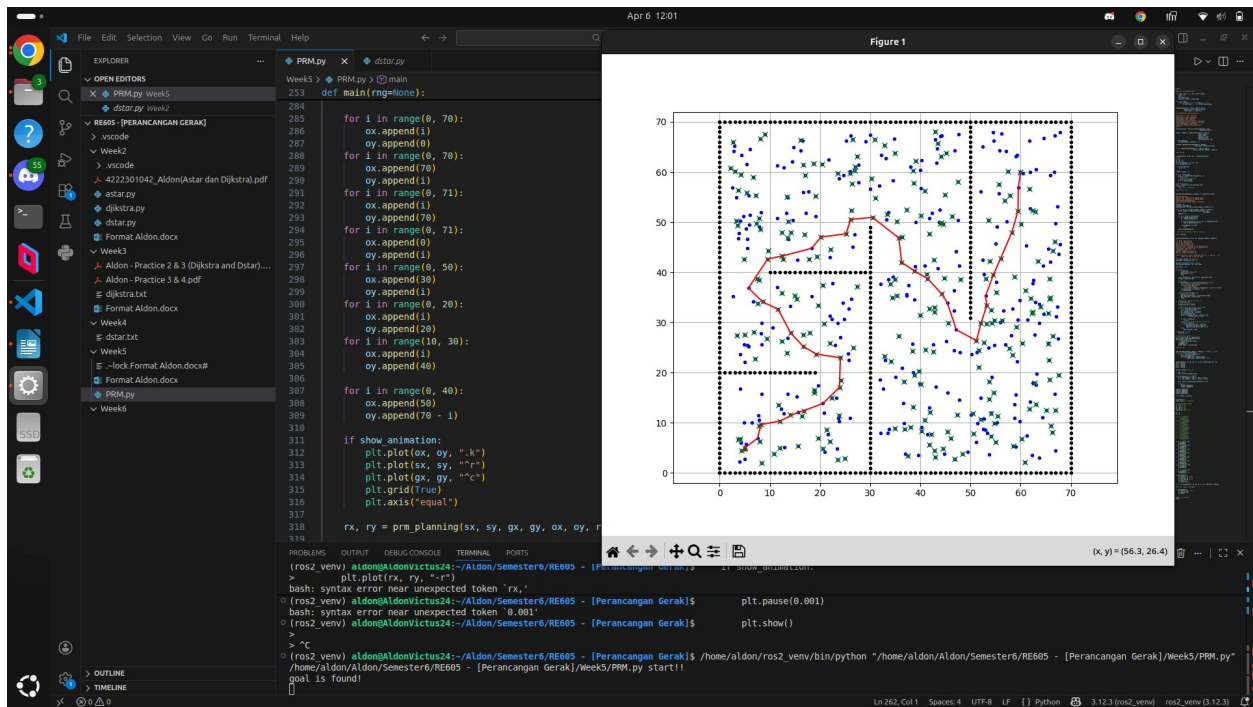
Fase 2

Menjalankan logika Dijkstra dari start ke finish melalui scatter point atau titik acuan yang saling terhubung.

Berikut adalah screenshot saya mencoba algoritma ini:



Dapat dilihat dari kedua gambar diatas, di kiri adalah fase 1 yang dimana titik scatter point (biru) di selimuti dan di sambung oleh silang Hijau yang dimana hijau ini menandakan suatu scatter point dengan scatter point berikutnya dapat terhubung dengan baik tanpa halangan. Lalu di gambar dikanan yaitu fase ke 2 setiap silang hijau yang sudah mengkonfirmasi titik acuan akan dijadikan tumpuan algoritma Dijkstra dan algoritma akan membikin jalan dari start ke finish berdasarkananya.



Diatas adalah hasil percobaan saya setelah merombak bentuk area map sesuai dengan perintah pada **Practice #4 PRM** dan disini jika saya menggunakan kode original yaitu yang dimana ukuran robot = 5.0 maka robot akan terlalu besar dan logika akan error karena ada area yang dimana ukuran itu tidak cukup untuk robot bermanuver.

```

# start and goal position
sx = 5.0 # [m]
sy = 5.0 # [m]
gx = 60.0 # [m]
gy = 60.0 # [m]
robot_size = 2.0 # [m]
  
```

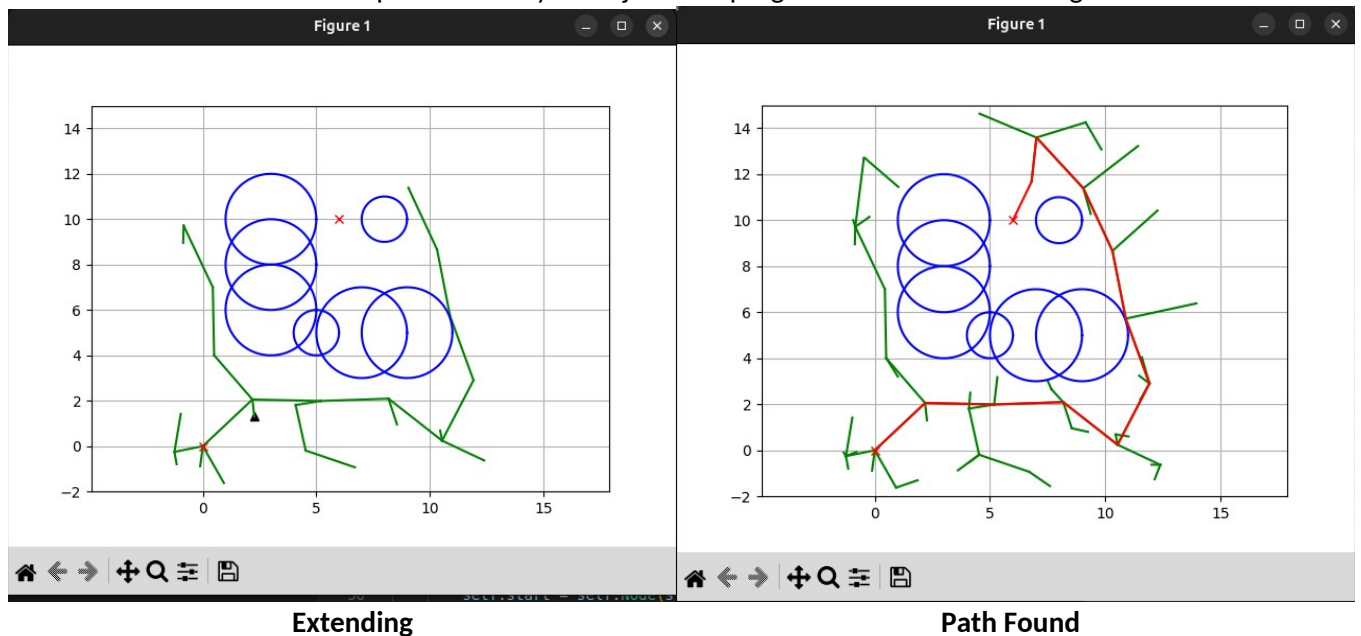
Diatas adalah bagian kode yang harus di ganti dimana saya memposisikan ulang start dan goal coordinate dan juga menurunkan ukuran robot dari 5 ke 2.

Dan untuk menjawab pertanyaan terakhir dari **Practice #4** tentang bagaimana meningkatkan efisiensi dan kualitas dari program PRM ini, menurut saya PRM ini dapat di dubah bagian pengolahan data node-to-node nya yang dimana sekarang di kode ini menggunakan Dijkstra dan diubah ke A* untuk meningkatkan kecepatan algoritma karena A* dapat menentukan jalur tercepat karena A* menggunakan Heuristik.

PRACTICE - #5 [RRT]

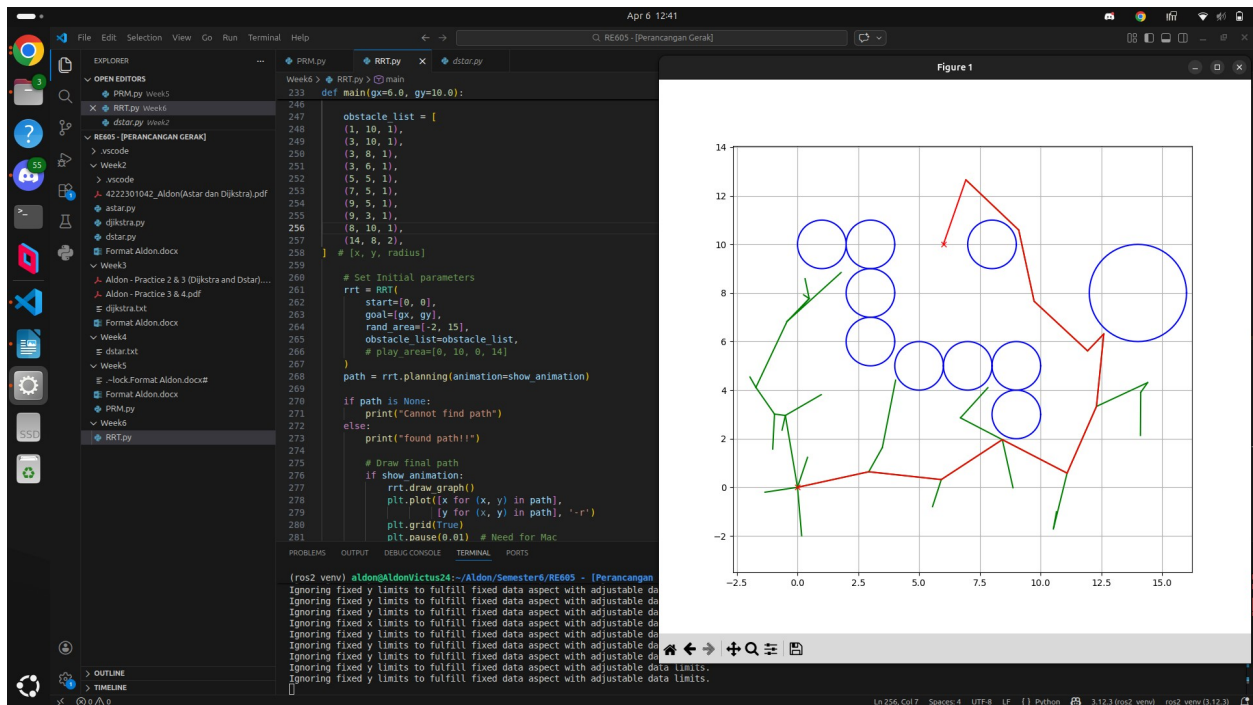
RRT atau **Rapidly-Exploring Random Trees** adalah algoritma yang dimana alur kerjanya logika yang menumbuhkan sebuah path yang memiliki bentuk serupa dengan sebuah pohon. Alur ini bekerja hanya dengan melakukan 3 hal yaitu **Memilih titik random di space (di area manapun)** lalu **Mencari dan memilih node atau titik phon terdekat yang sudah ada** dan **memperpanjang titik pohon itu sepanjang step size yang sudah di atur**. Algoritma ini akan terus mengulangi 3 hal ini sampai dia menemukan titik goal positionnya.

Dibawah ini adalah Screenshot percobaan saya menjalankan program tertera di E-Learning:



Dapat dilihat di gambar di atas, saat algoritma sedang mencari titik finish, dia akan selalu memilih ujung cabang suatu jalur (pohon) dan memperpanjangnya, terakhir di gambar sisi kanan ketika titik finish sudah ditemukan maka algoritma akan membuat jalur tercepat dari semua cabang yang ada dari titik start ke finish.

Adapaula tuags yangdiberikan ke saya di E-Learning tentang Practice #5 yang dimana saya harus mengubah tata letak arena dan menjalankan kembali algoritma ini, hasil dari pengerjaan saya dapat dilihat di bawah ini:



Diatas adalah hasil running program RRT saya setelah mengubah tata letak obstacle sesuai dengan gambar yang ada di penugasan **Practice#5**.

Setelah menjalankan program ini. Saya mempelajari bahwa kita perlu melakukan Collision Check pada map karena pada salah satu Run saya, sebuah cabang jalur Tree menembus melalui kedua lingkaran dan mencapai titik finish dengan cepat yang terkesan curang.

Lalu Apa yang dimaksud dengan “Ekstraksi Jalur”. Ekstraksi jalur adalah proses traceback algoritma tree dari titik finish ke start untuk membuat path nya setelah algoritma RRT menemukan titik goal.

Lalu membandingkan PRM dan RRT. Menurut saya PRM jauh lebih unggul karena PRM membuat roadmap terlebih dahulu lalu meng-scan atau menyelimutinya dengan algoritma konfirmasi seperti yang saya jelaskan tadi lalu menjalankan algoritma tambahan seperti Dijkstra atau A* untuk memproses path planningnya dan ini bisa dilakukan untk multiple goal sekaligus. Sedangkan RRT harus membuat tree setiap kali hanya untuk mencari satu goal dan proses pencarian ini sendiri dapat memerlukan waktu lama karena titik cabang pohon yang di lakukan random dan tidak begitu reliable.